



TÉCNICO SUPERIOR EN DESARROLLO DE APLICACIONES
MULTIPLATAFORMA
Departamento de Informática

PROYECTO



Eventvs Mérida

Manual Técnico

Autor/es: David Muñoz Collado, Adrián Pérez Morales, Eva Retamar Muñoz
Curso Académico: 2025/2026
Tutor Proyecto : María Francisca Roncero Holgado

Índice

Índice	2
1. Introducción	3
Descripción del proyecto	3
2. Objetivos	3
3. Tecnologías involucradas	4
Lenguajes / Frameworks utilizados:	4
4. Proceso de desarrollo	6
4.1. Análisis	6
4.2. Desarrollo (diagrama de secuencias,...)	13
4.3 Desarrollo	13
4.4 Prototipado en Figma	16
4.5 Arquitectura del repositorio	16
4.6 Desarrollo Iterativo	17
4.7 Pruebas realizadas	18
5. Bibliografía	22

1. Introducción

Descripción del proyecto

Eventvs Mérida es una aplicación móvil diseñada para centralizar la información sobre los eventos culturales y de ocio que se celebran en la ciudad de Mérida. Actualmente, gran parte de esta información se encuentra dispersa en diferentes páginas web, redes sociales o medios locales, lo que dificulta que ciudadanos y turistas puedan conocer de forma sencilla todas las actividades disponibles.

La plataforma está dirigida tanto a ciudadanos de Mérida como a turistas que desean conocer la oferta cultural de la ciudad. A través de una interfaz sencilla e intuitiva, los usuarios pueden explorar los eventos disponibles, visualizar su información y conocer su ubicación.

2. Objetivos

Objetivos funcionales

- Centralizar la información de eventos culturales de Mérida en una única aplicación.
- Permitir a los usuarios consultar un listado actualizado de eventos.
- Mostrar la información detallada de cada evento.
- Visualizar la ubicación de los eventos mediante un mapa.
- Permitir el registro y autenticación de usuarios en la aplicación.

Objetivos técnicos

- Desarrollar una aplicación móvil utilizando tecnologías actuales de desarrollo.
- Implementar un sistema de gestión de usuarios mediante autenticación.
- Integrar una base de datos en la nube para almacenar la información de eventos.
- Aplicar buenas prácticas de programación y diseño de interfaces.

3. Tecnologías involucradas

Se deben detallar qué tecnologías se utilizarán durante del desarrollo, incluyendo un marco teórico en caso de tecnologías que no se hayan utilizado a lo largo del ciclo (diseño de la base de datos, diagrama de clases, lenguajes de desarrollo, frameworks, lenguajes, etc).

Lenguajes / Frameworks utilizados:

Java  <i>Figura 1: Logo de Java</i> Lenguaje principal utilizado en el backend para la construcción de la API.	Spring Boot  <i>Figura 2: Logo de Spring Boot</i> Junto a Java son la base de nuestra API gracias a todas las funcionalidades que ofrece el framework.
Flutter  <i>Figura 3: Logo de Flutter</i> Framework principal utilizado para el frontend de la aplicación	Python  <i>Figura 4: Logo de Python</i> Lenguaje utilizado para la programación de los diferentes scripts que utilizamos para obtener eventos

Angular  <i>Figura 5: Logo de Angular</i> Lenguaje principal utilizado para la landing page	SCSS  <i>Figura 6: Logo de SCSS</i> Lenguaje que junto a Angular, lo utilizamos para darle estilos a la landing page
TypeScript  <i>Figura 7: Logo de TypeScript</i> Lenguaje que junto a los 2 anteriores definimos la lógica de la landing page	HTML 5  <i>Figura 8: Logo de HTML5</i> Lenguaje de marcas utilizado en el panel de administración
CSS  <i>Figura 9: Logo de CSS</i> Junto a HTML lo utilizamos para darle estilos al panel de administración	JS  <i>Figura 10: Logo de JS</i> Junto a HTML y CSS lo utilizamos para la lógica y comunicación con la API en el panel de administración

Bootstrap



Figura 11: Logo de bootstrap

Framework que combinamos con los 3 lenguajes anteriores para darle un estilo profesional al panel de administración

4. Proceso de desarrollo

4.1. Análisis

4.1.1 Diagrama E-R

Este diagrama muestra las diferentes relaciones entre las tablas de la base de datos así como los datos de cada uno de ellos.

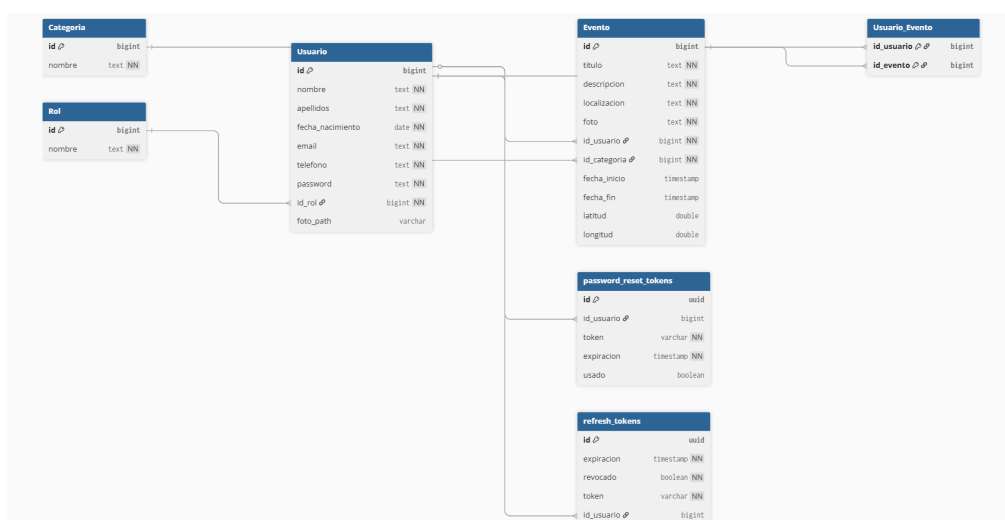


Figura 12: Diagrama E-R

1. Usuario

La entidad Usuario almacena la información de las personas registradas en la plataforma. Estos usuarios pueden consultar eventos, participar en ellos o, dependiendo de su rol, gestionarlos.

Campos principales:

- id: Identificador único del usuario.
- nombre: Nombre del usuario.
- apellidos: Apellidos del usuario.
- fecha_nacimiento: Fecha de nacimiento.
- email: Correo electrónico del usuario, usado para acceso o contacto.
- teléfono: Número de teléfono del usuario.
- password: Contraseña para autenticación en la plataforma.
- id_rol: Referencia al rol del usuario dentro del sistema.

Función en el sistema:

Permite gestionar el registro, autenticación y datos personales de los usuarios que utilizan la aplicación.

2. Rol

La entidad Rol define los diferentes tipos de usuarios dentro de la plataforma.

Campos principales:

- id: Identificador único del rol.
- nombre: Nombre del rol (por ejemplo: administrador, organizador, usuario).

Función en el sistema:

Permite establecer niveles de permisos y controlar qué acciones puede realizar cada tipo de usuario dentro de la aplicación.

3. Evento

La entidad Evento almacena toda la información relacionada con los eventos culturales disponibles en la plataforma.

Campos principales:

- id: Identificador único del evento.
- título: Nombre del evento.
- descripción: Información detallada sobre el evento.
- fecha_hora: Fecha y hora en la que se celebra.
- localización: Lugar donde se realiza el evento.
- foto: Imagen representativa del evento.
- id_usuario: Usuario que creó o gestiona el evento.
- categoría: Categoría a la que pertenece el evento.

Función en el sistema:

Permite registrar y mostrar todos los eventos culturales disponibles en la ciudad, facilitando su consulta por parte de los usuarios.

4. Categoría

La entidad Categoría permite clasificar los eventos según su tipo o temática.

Campos principales:

- id: Identificador único de la categoría.
- nombre: Nombre de la categoría (por ejemplo: conciertos, teatro, exposiciones, festivales).

Función en el sistema:

Facilita la organización y filtrado de eventos, permitiendo a los usuarios buscar actividades según sus intereses.

5. Usuario_Evento

La entidad Usuario_Evento es una tabla intermedia que representa la relación entre usuarios y eventos.

Campos principales:

- id_usuario: Identificador del usuario.
- id_evento: Identificador del evento.

Función en el sistema:

Permite registrar la participación o relación de los usuarios con los eventos, como por ejemplo:

- eventos guardados en la agenda personal
- eventos a los que el usuario asistirá
- favoritos o recordatorios

Esta tabla gestiona una relación muchos a muchos, ya que:

- un usuario puede participar en varios eventos
- un evento puede tener muchos usuarios interesados.

6. password_reset_tokens

La entidad password_reset_tokens permite generar tokens para poder restablecer la contraseña.

Campos principales:

- id: Identificador único del token generado.
- id_usuario: Identificador del usuario el cuál ha solicitado el restablecimiento de su contraseña.
- token: Token generado para validar el cambio de contraseña.
- expiracion: Datetime con la fecha de expiración del token (30 minutos después de la creación)
- usado: Valor booleano que controla si el token ya ha sido utilizado o no.

Función en el sistema:

Permite gestionar un sistema con autenticación del restablecimiento de contraseñas para evitar manipulaciones indebidas de las contraseñas.

7. refresh_tokens

La entidad refresh_tokens permite refrescar la cookie o "JSESSIONID" de un usuario logueado para mantener su sesión iniciada y que pueda realizar acciones sobre los diferentes endpoints que tenga permiso.

Campos principales:

- id: Identificador único del token generado.

- expiracion: Datetime con la fecha de expiración de la cookie (2 horas después de la creación)
- revocado: Valor booleano que controla si la cookie ya ha finalizado su tiempo de vida.
- token: Token generado para la cookie.
- id_usuario: Identificador del usuario el cuál se restablece la cookie.

Función en el sistema:

Permite gestionar un sistema de refresco de la cookie de un usuario autenticado en la app.

4.1.2 Diagramas de casos de uso

4.1.2.1 Diagrama de usuario no registrado

Este diagrama muestra de forma resumida las funcionalidades disponibles que puede realizar un usuario que no se ha registrado en la aplicación.

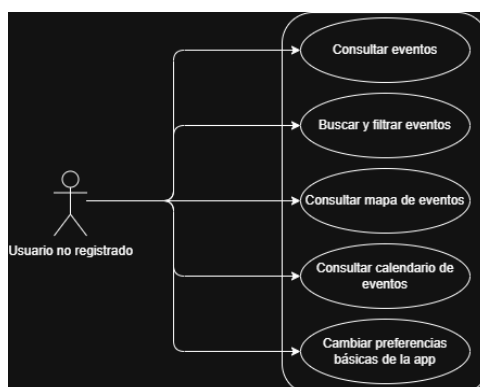


Figura 13: Diagrama de caso de uso de usuario no registrado

4.1.2.2 Diagrama de usuario registrado

Este diagrama muestra de forma resumida las funcionalidades que puede realizar un usuario registrado.

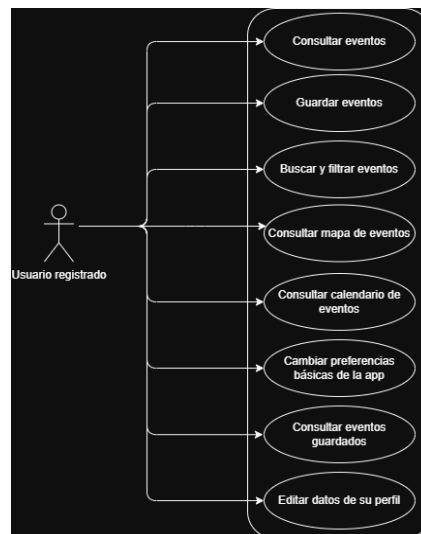


Figura 14: *Diagrama de caso de uso de usuario registrado*

4.1.2.3 Diagrama de organizador

Este diagrama muestra de forma resumida las funcionalidades del organizador.

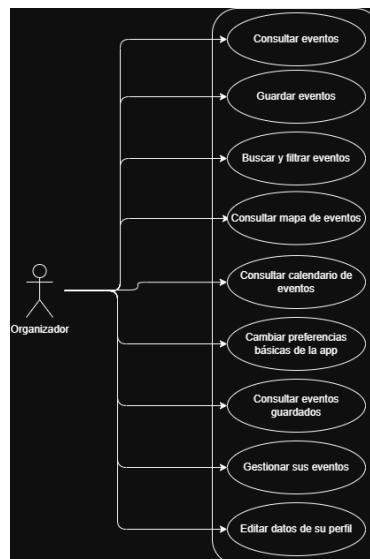


Figura 15: *Diagrama de caso de uso de organizador*

4.1.2.4 Diagrama de administrador

Este diagrama muestra de forma resumida las funcionalidades del administrador.

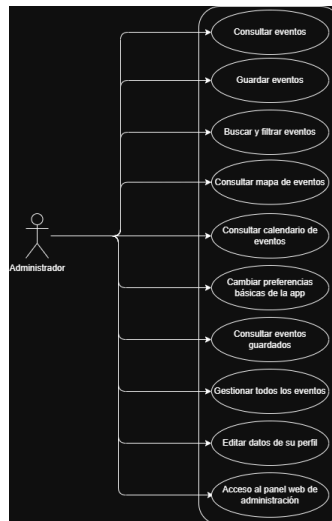


Figura 16: Diagrama de caso de uso de administrador

4.1.3 Diagrama de flujo

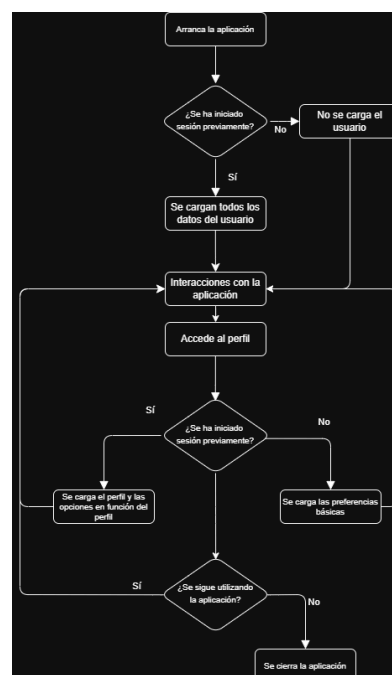


Figura 18: Diagrama de flujo

4.2. Desarrollo (diagrama de secuencias,...)

4.2.1 Diagrama de secuencias

Este diagrama muestra cómo funcionan todas las comunicaciones/acciones de la aplicación.

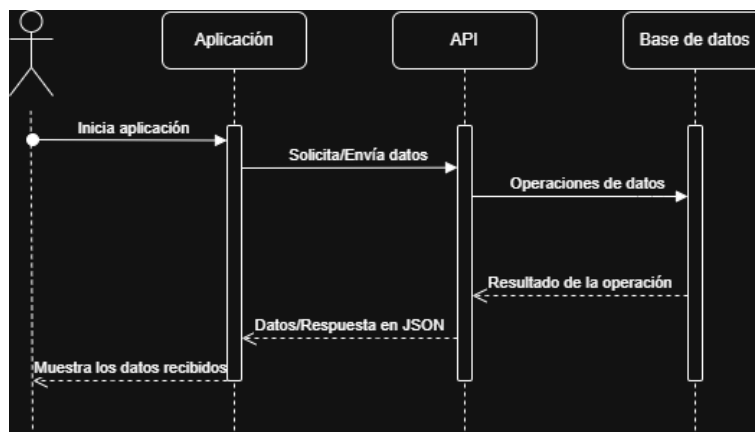


Figura 17: Diagrama de secuencias

4.3 Desarrollo

4.3.1 Planificación y Definición de Requisitos

La fase inicial consistió en identificar el problema a resolver: la fragmentación de la información cultural y de ocio de Mérida. A partir de este diagnóstico se definieron los requisitos principales del sistema:

- Consulta de eventos culturales y de ocio en Mérida.
- Filtrado por categoría, fecha y ubicación.
- Visualización de detalle de cada evento.
- Diseño accesible y atractivo para distintos perfiles de usuario (ciudadanos y turistas).
- Arquitectura cliente-servidor con API REST.

La definición del alcance se materializó en un prototipo elaborado en Figma, que sirvió de guía visual para basarnos a lo largo del desarrollo.

4.3.2 Diseño de la Arquitectura

La arquitectura del sistema de Eventvs Mérida se ha diseñado siguiendo una estructura en capas, con el objetivo de separar de forma clara la interfaz de usuario, la lógica de negocio el

acceso a los datos. Esta organización facilita el mantenimiento del proyecto, mejora su escalabilidad y permite que cada parte del sistema evolucione de forma independiente.

La solución está compuesta por una aplicación móvil desarrollada en Flutter, un backend implementado con Spring Boot y una base de datos gestionada mediante Supabase. Esta distribución permite construir un sistema modular en el que el frontend se encarga de la interacción con el usuario, el backend de la lógica de negocio y la exposición de servicios, y la capa de datos del almacenamiento y recuperación de la información.

4.3.2.1 Patrón de Arquitectura

El proyecto adopta una arquitectura en capas con un enfoque próximo a Clean Architecture, separando las responsabilidades principales del sistema en tres niveles: presentación, lógica de negocio y datos.

Esta elección se ha realizado porque permite:

- Mantener el código más organizado y modular.
- Separar la interfaz visual de la lógica interna de la aplicación.
- Facilitar el mantenimiento y la ampliación futura del sistema.
- Mejorar la reutilización de componentes.
- Simplificar las pruebas y la detección de errores.

En el caso de Eventvs Mérida, este enfoque resulta adecuado porque la aplicación no solo muestra información, sino que también necesita gestionar usuarios, consultar eventos, comunicarse con un backend y acceder a servicios externos. Por ello, una arquitectura en capas ofrece una solución más escalable que otros patrones más simples como MVC.

4.3.2.2 Diagrama de Arquitectura

La arquitectura general del sistema puede representarse de la siguiente forma:

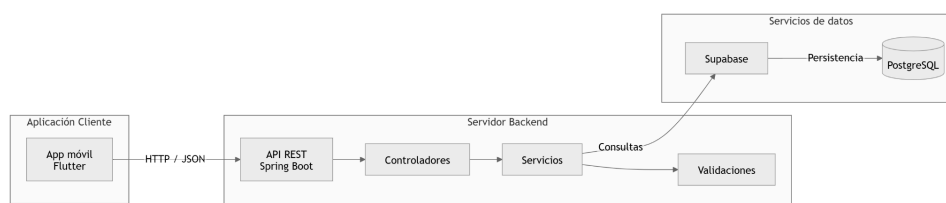


Figura 18: Diagrama de arquitectura

4.3.2.3 Componentes Principales

El sistema se organiza en tres capas principales: presentación, lógica de negocio y datos. Cada una de estas capas tiene responsabilidades específicas que permiten mantener una arquitectura modular y fácil de mantener.

Capa de presentación

La capa de presentación está formada por la aplicación móvil desarrollada en Flutter. Su función principal es mostrar la información al usuario y gestionar la interacción con la interfaz.

Entre los elementos principales de esta capa se encuentran:

- Pantallas de la aplicación (login, registro, listado de eventos, detalle de evento, mapa, etc.).
- Widgets reutilizables que componen la interfaz gráfica.
- Navegación entre pantallas.
- Formularios de entrada de datos.

Esta capa se encarga de recoger las acciones del usuario y enviar las peticiones correspondientes al backend a través de la API, pero no implementa la lógica principal del sistema.

Capa de lógica de negocio

La capa de lógica de negocio se encuentra implementada en el backend desarrollado con Spring Boot. Esta capa se encarga de procesar la información recibida desde la aplicación móvil, aplicar las reglas del sistema y coordinar el acceso a los datos.

Entre sus responsabilidades se encuentran:

- Gestión de usuarios y autenticación.
- Validación de datos recibidos desde la aplicación.
- Procesamiento de peticiones relacionadas con eventos.
- Implementación de reglas de negocio del sistema.
- Exposición de una API REST que permite la comunicación entre la aplicación móvil y el backend.

Esta capa actúa como intermediaria entre la interfaz de usuario y la base de datos.

Capa de datos

La capa de datos es la responsable del almacenamiento y recuperación de la información utilizada por el sistema. En este proyecto se utiliza Supabase, que proporciona una base de datos basada en PostgreSQL.

Los principales componentes de esta capa son:

- Base de datos PostgreSQL, donde se almacenan los eventos, usuarios y demás información de la aplicación.
- Repositorios de acceso a datos, utilizados por el backend para consultar o modificar la información.
- Servicios de base de datos proporcionados por Supabase, que facilitan la gestión de datos y autenticación.

Esta capa permite persistir la información del sistema y garantizar que los datos puedan ser consultados o actualizados de forma segura por el backend.

4.4 Prototipado en Figma

Antes de comenzar el desarrollo de la interfaz, elaboramos un prototipo completo en Figma. Este paso fue fundamental para:

- Acordar la identidad visual del producto (paleta de colores, tipografía, iconografía).
- Validar los flujos de navegación principales antes de escribir código.
- Servir como referencia de diseño durante la implementación en Flutter.
- Reducir el número de revisiones de UI durante el desarrollo.

El prototipo refleja las pantallas principales: listado de eventos, detalle de evento, filtros y pantalla de inicio.

4.5 Arquitectura del repositorio

El repositorio se encuentra alojado en GitHub. La estructura de directorios adoptada separa claramente las distintas partes del proyecto:

- /app** → Código de la app en Flutter y backend en Spring Boot.
- /docs** → Documentación del proyecto.

/scripts → Scripts en Python auxiliares y de automatización.

/web → Código fuente de las 3 webs del proyecto.

4.6 Desarrollo Iterativo

El desarrollo se organizó en iteraciones cortas, abordando funcionalidades de forma incremental. Las principales áreas de trabajo en cada iteración fueron:

4.6.1 Backend (Spring Boot / Java):

- Definición del modelo de datos de eventos, categorías y ubicaciones.
- Implementación de endpoints REST para consulta y filtrado de eventos.
- Configuración de la capa de persistencia.
- Pruebas de integración de la API.

4.6.2 Frontend móvil (Flutter / Dart):

- Configuración inicial del proyecto Flutter y estructura de carpetas.
- Implementación de las pantallas principales siguiendo el prototipo Figma.
- Integración con la API REST del backend mediante peticiones HTTP.
- Gestión de estado y navegación entre pantallas.
- Adaptación del diseño a distintos tamaños de pantalla.

4.6.3 Landing Page (Angular / SCSS / Typescript):

- Maquetación y diseño responsivo de la página de presentación.
- Implementación de secciones: descripción del producto, capturas, equipo y enlace de descarga.

4.6.4. Panel de administración (HTML / CSS / JS / Bootstrap)

- Maquetación y diseño de las diferentes páginas del panel.
- Conexión de cada acción de cada página con los endpoints de la API.

4.6.5 Web de recuperación de contraseña (HTML / CSS / JS / Bootstrap)

- Maquetación y diseño del formulario de recuperación de contraseña.
- Implementación de la lógica para el restablecimiento de la contraseña.
- Configuración del envío de correos electrónicos para poder acceder a la web con un token válido.

4.6.6 Scripts auxiliares(Python):

- Automatización de tareas como la subida de eventos de algunas fuentes.
- Procesamiento y preparación de datos para poblar la base de datos.

4.7 Pruebas realizadas

Las pruebas que hemos realizado han sido de carácter manual. adoptando un enfoque continuo e incremental: cada vez que se completaba el desarrollo de una funcionalidad, probamos todos los posibles casos de uso antes de integrar el código en la rama principal.

Esto nos permitió detectar errores cuando el código aún es reciente y es más fácil localizarlo, evita la acumulación de errores evitando que se mezclaran con nuevas funcionalidades manteniendo una coherencia y rigurosidad al código.

P-01. Modelo de datos y CRUD básicos	
Momento de prueba	Al definir las entidades (Evento, Categoría, Ubicación, etc)
Tipo de prueba	Prueba de integración manual del backend.
Qué se verificó	Los endpoints de creación de registros devolvían respuesta 201 con los datos correctamente persistidos. Los endpoints de consulta devolvían los registros almacenados en formato JSON esperado. Los errores de validación (campos obligatorios vacíos, tipos incorrectos) devolvían códigos de error adecuados (400). Los scripts Python de carga de datos poblaban la base de datos sin errores.
Criterio de éxito	La API respondía correctamente a las operaciones CRUD básicas sobre las entidades del dominio.

P-02. API REST — Endpoints de consulta de eventos	
Momento de prueba	Al implementar los endpoints de listado y detalle de eventos de forma paginada.
Tipo de prueba	Prueba funcional manual haciendo uso de Swagger.
Qué se verificó	GET /eventos devolvía el listado completo de eventos con la estructura JSON correcta. GET /eventos/{id} devolvía el detalle de un evento existente. GET /eventos/{id} con un id inexistente devolvía error 404. Los campos de respuesta (título, descripción, fecha, categoría, ubicación) se correspondían con los datos almacenados. La paginación del listado funcionaba correctamente al variar los parámetros page y size.

Criterio de éxito	Todos los endpoints respondían con los datos y códigos HTTP correctos en los escenarios positivos y negativos probados.
--------------------------	---

P-03. API REST — Filtrado de eventos

Momento de prueba	Al implementar los parámetros de filtrado por categoría, fecha y ubicación.
Tipo de prueba	Prueba funcional manual con herramienta de test de APIs.
Qué se verificó	El filtro por categoría devolvía únicamente eventos de la categoría indicada. El filtro por rango de fechas devolvía solo eventos dentro del intervalo especificado. La combinación de varios filtros simultáneos producía resultados coherentes. Una consulta con filtros que no coincidían con ningún evento devolvía una lista vacía (no un error).
Criterio de éxito	El sistema filtraba correctamente los eventos según los parámetros recibidos, incluyendo combinaciones de filtros.

P-04. Desarrollo de las pantallas/componentes de la app (Flutter)

Momento de prueba	A medida que se implementaban las pantallas: listado de eventos , detalle de eventos, mapa, calendario, perfil, etc.
Tipo de prueba	Prueba de interfaz manual en emulador Android y, cuando fue posible, en dispositivo físico.
Qué se verificó	La pantalla de inicio se cargaba correctamente y mostraba los elementos de navegación esperados. El listado de eventos se mostraba con los datos reales obtenidos de la API. Cada tarjeta de evento mostraba correctamente título, fecha, categoría e imagen. La pantalla de detalle se abría al pulsar sobre un evento y presentaba toda la información completa. La app mostraba un indicador de carga (loading) mientras esperaba la respuesta del servidor. Ante una respuesta de error del servidor, la app mostraba un mensaje de error en lugar de bloquearse.
Criterio de éxito	La navegación entre pantallas era fluida, los datos se cargaban desde la API y el diseño era consistente con el prototipo.

P-05. Integración app móvil — backend (consumo de la API)

Momento de prueba	Al conectar la aplicación Flutter con los endpoints reales del backend.
Tipo de prueba	Prueba de integración manual end-to-end.
Qué se verificó	La app obtenía y mostraba correctamente los datos devueltos por la API en el listado de eventos. Los filtros aplicados en la interfaz se traducían en peticiones correctas a la API y los resultados mostrados eran los esperados. La app mostraba un indicador de carga (loading) mientras esperaba la respuesta del servidor. Ante una respuesta de error del servidor, la app mostraba un mensaje de error en lugar de bloquearse. El comportamiento era correcto tanto con backend local (desarrollo) como con la URL de despliegue.
Criterio de éxito	El flujo completo usuario → app → API → base de datos → respuesta funcionaba de extremo a extremo sin errores visibles.

P-06. Funcionalidad de filtrado en la interfaz de usuario

Momento de prueba	Al implementar los controles de filtrado en la app Flutter (selección de categoría, fechas, etc.).
Tipo de prueba	Prueba funcional manual en emulador y dispositivo físico.
Qué se verificó	Los controles de filtrado se mostraban correctamente en la interfaz. Al seleccionar un filtro, la lista de eventos se actualizaba mostrando solo los resultados correspondientes. Al eliminar o resetear un filtro, la lista volvía a mostrar todos los eventos. La combinación de varios filtros producía los resultados correctos. La interacción con los controles era fluida y no generaba retardos perceptibles.
Criterio de éxito	El sistema de filtrado funcionaba de forma correcta y ofrecía una experiencia de usuario ágil y coherente.

P-07. Panel web de administración

Momento de prueba	Al desarrollar las pantallas del panel de administración.
Tipo de prueba	Prueba visual manual en navegadores y en dispositivos móviles.

Qué se verificó	Todas páginas del panel se renderizaban correctamente y se conectaban con los diversos endpoints de la API.. Los enlaces de navegación interna funcionaban correctamente. El despliegue automático en Vercel reflejaba los últimos cambios del repositorio. Las imágenes y recursos se cargaban sin errores.
Criterio de éxito	El panel era completamente funcional en las conexiones con la API y visualmente consistente en todos los navegadores.

P-08. Landing page

Momento de prueba	Al desarrollar las distintas secciones de la página web informativa.
Tipo de prueba	Prueba visual manual en navegadores y en dispositivos móviles.
Qué se verificó	Todas las secciones de la página se renderizaban correctamente. El diseño era responsivo y se adaptaba a pantallas móviles y de escritorio. Los enlaces de navegación interna funcionaban correctamente. El despliegue automático en Vercel reflejaba los últimos cambios del repositorio. Las imágenes y recursos estáticos se cargaban sin errores.
Criterio de éxito	La landing page era completamente funcional y visualmente consistente en todos los navegadores y dispositivos probados.

F-09. Web de recuperación de contraseñas

Momento de prueba	Al desarrollar el formulario de reseteo de la contraseña.
Tipo de prueba	Prueba visual manual en navegadores y en dispositivos móviles.
Qué se verificó	Todas los campos del formulario se renderizaban correctamente. El diseño era responsivo y se adaptaba a pantallas móviles y de escritorio. La lógica con el token a la hora de hacer el reseteo de contraseña es válida y funcional. El despliegue automático en Vercel reflejaba los últimos cambios del repositorio. Las imágenes y recursos estáticos se cargaban sin errores.

Criterio de éxito	
	La web de recuperación de contraseña era completamente funcional y visualmente consistente en todos los navegadores y dispositivos probados.

5. Bibliografía

Se deben incluir la referencias a todos los recursos utilizados en el desarrollo del proyecto, tanto libros, manuales, etc como bibliografía web.

- Documentación oficial de Angular: <https://angular.dev/overview>
- Documentación oficial de Bootstrap: <https://getbootstrap.com/>
- Documentación oficial de Flutter: <https://docs.flutter.dev/>
- Documentación oficial de Python: <https://docs.python.org/3/>
- Documentación oficial de Spring Boot: <https://docs.spring.io/spring-boot/index.html>
- Documentación oficial de Supabase: <https://supabase.com/docs>
- Documentación Tutorial Coach Mark: https://pub.dev/packages/tutorial_coach_mark
- Resolución de dudas respecto a JS: <https://stackoverflow.com/questions>